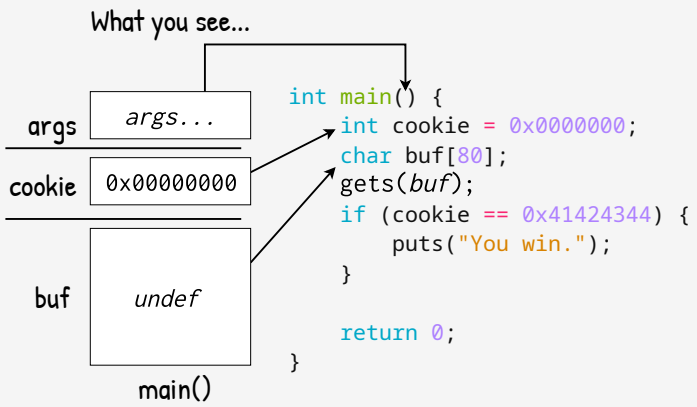
Exploit:
Buffer Overflow, ROP, NX, CFI, Canary

Taller de Seguridad

Ing. Martin Di Paola - Fiuba

Can you print *"You win"*?

```c
int main() {
    int cookie = 0x0000000;
    char buf[80];
    gets(buf);
    if (cookie == 0x41424344) {
        puts("You win.");
    }

    return 0;
}
```

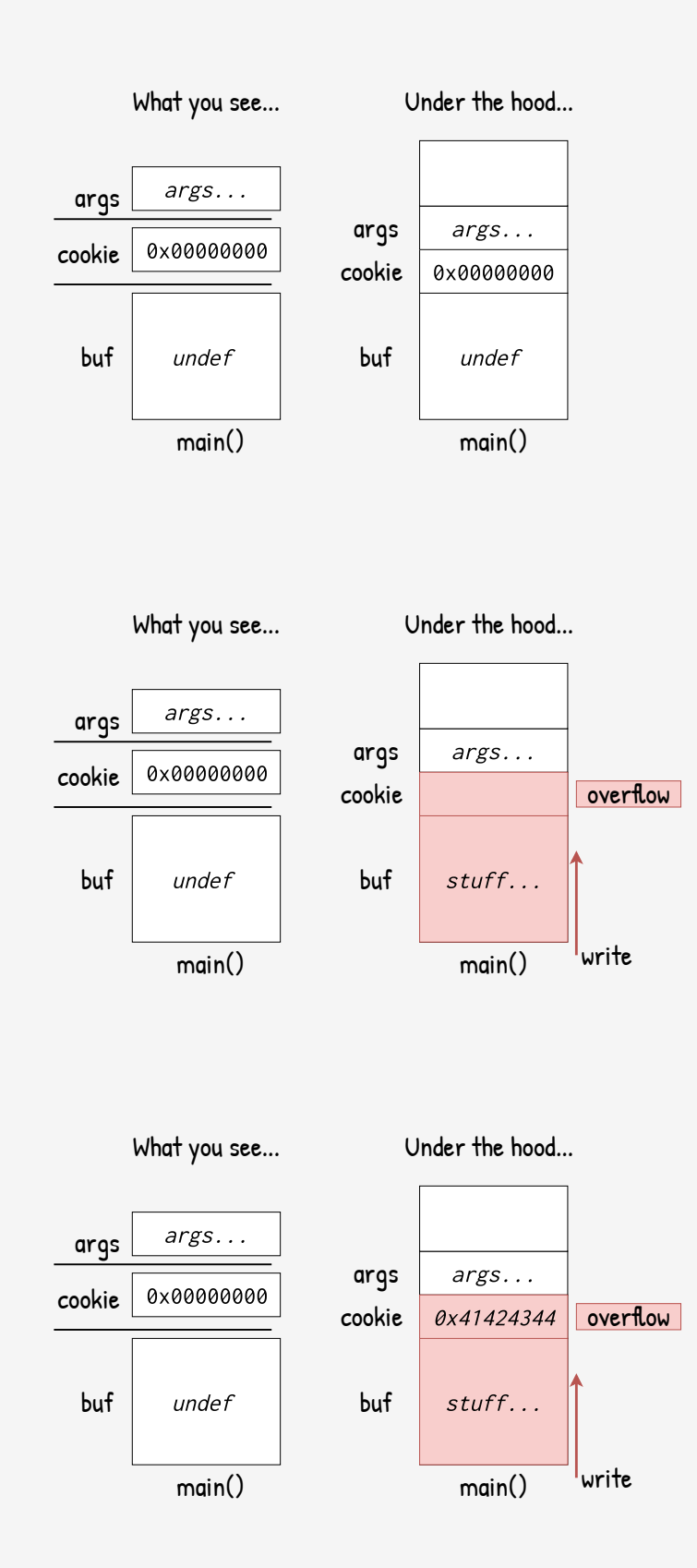


Observa el código C. Qué hace?

Es posible tener en `cookie` el valor `0x41424344` y q el programa imprima `"You win"`?

A nivel de la máquina abtracta de C (lo que el estandar te dice), en memoria tenes los argumentos de la función y sus variables locales.

Es lo que uno ve como programador.



Pero no existe ninguna máquina abstracta. En realidad, los compiladores y el runtime definen todos los detalles de implementación.

Detras de escena, hay un único espacio *continuo* de memoria donde los argumentos y variables locales son guardados.

Y aca está la clave: como `gets` escribe en `buf` sin tener un límite, basta con que `gets` escriba mucho más de lo que `buf` puede almacenar, pisando memoria continua, q en nuestro caso es la variable local `cookie`.

Esto es el clásico *buffer overflow*.

Entonces, al final sí se puede imprimir `"You win"`.

Can you print *"You win"*?

Can you print whatever you like?

↓

Can you take control of the program?

↓

Can you take control of the host?

↓

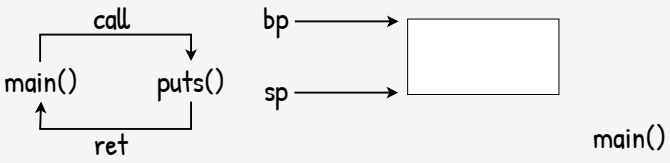
Can you ***truly win***?

```c
int main() {
    int cookie = 0x0000000;
    char buf[80];
    gets(buf);
    if (cookie == 0x41424344) {
        puts("You win.");
    }

    return 0;
}
```

Y q más se podría hacer?

Tomá esta variante de codigo. Podrías hacer que el programa abra el archivo `~/flag.txt` y lo lee para vos?

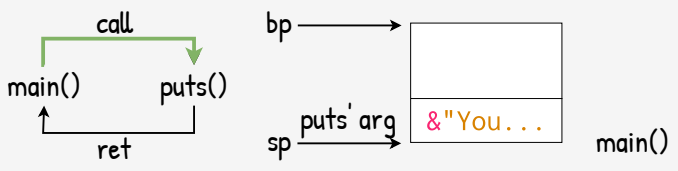
```c
int main() {
    int cookie = 0x0000000;
    char buf[80];
    gets(buf);
    if (cookie == 0x41424344) {
        puts("Your flag is ~/flag.txt.");
    }

    return 0;
}
```



Cuando una funcion llama a otra (como `main` llamando a `puts`), hay una serie de cambios en el stack.

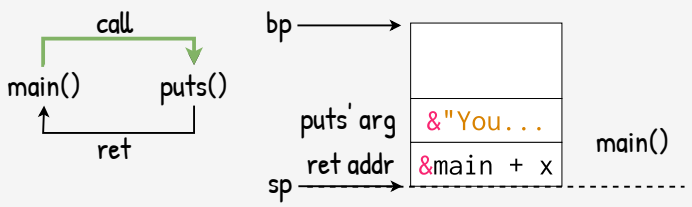
Inicialmente `main` tiene 2 registros, `bp` (base pointer) y `sp` (stack pointer) apuntando al comienzo del inicio de la porción del stack q le corresponde a `main` y al final (al top del stack).

Exactamente que registros depende del compiler y arquitectura, por ahora no nos metemos con ese nivel detalle.



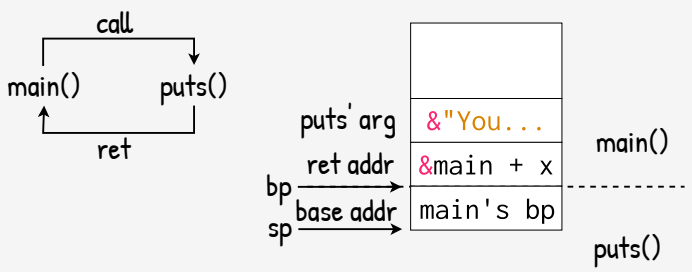

```
mov ecx &"You..."
push ecx
call &puts
```

Cuando se hace la llamada, primero `main` pone los argumentos en el stack.




```
mov ecx &"You..."
push ecx
call &puts
```

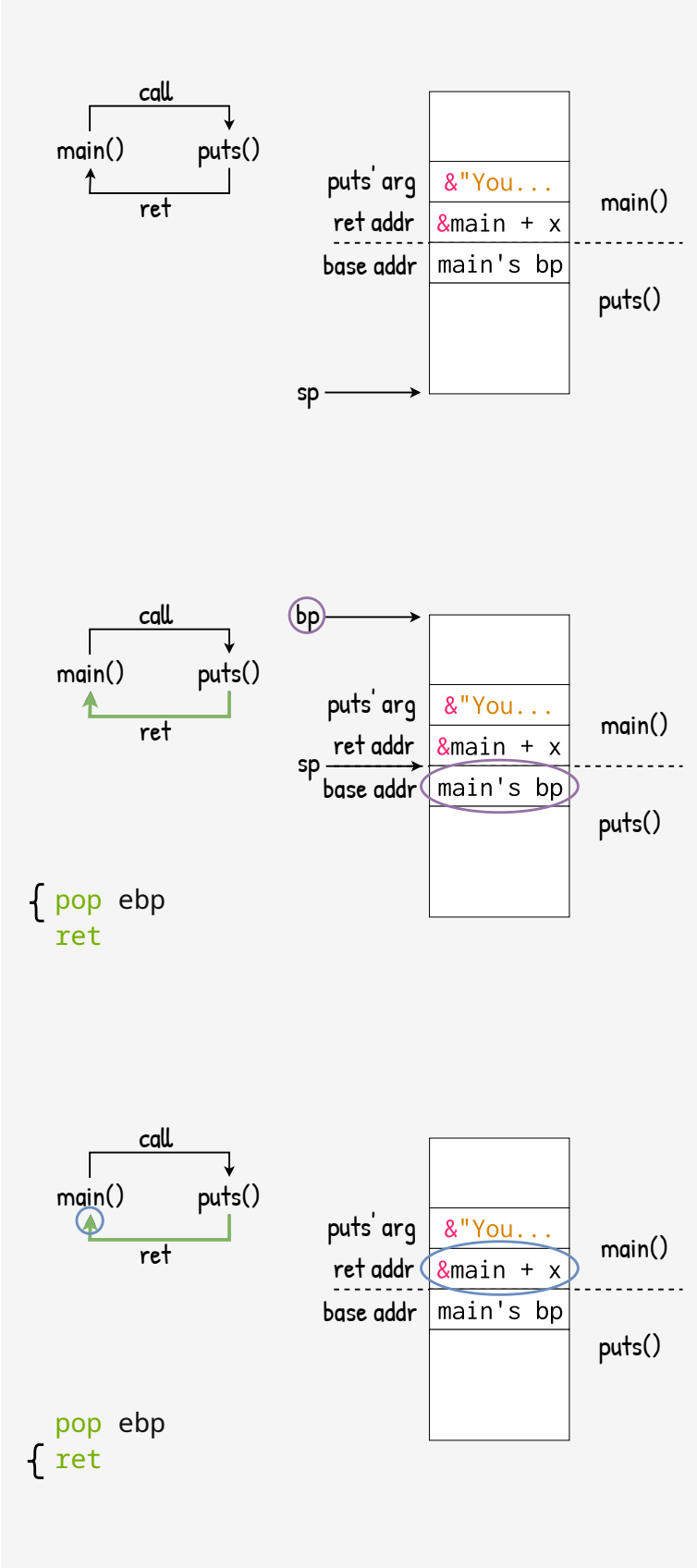
Luego hace un `call`. Esto hace que la dirección de retorno (`ret`) se guarde en el stack antes de que el programa deje de ejecutar `main` y pase a ejecutar `puts`.




```
push ebp
mov ebp esp
mov ecx esp-8
```

Luego el valor de `bp` es guardado en el stack y una nueva base se define para `puts`.

Esto es el *preludio* de `puts`.

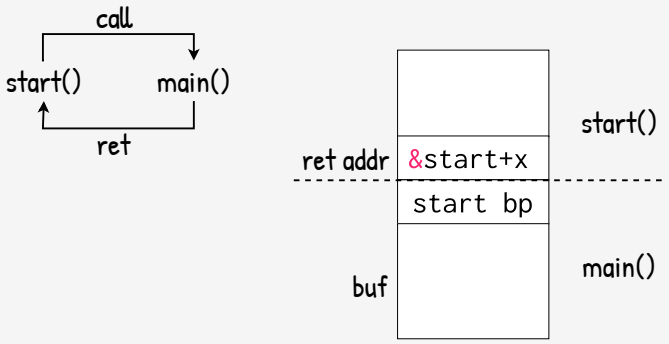


A partir de aquí más espacio del stack es reservado para las variables locales de `puts`.

Cuando `puts` termina, se lee del stack el `bp` de la funcion padre (`main`) para restaurar el registro.

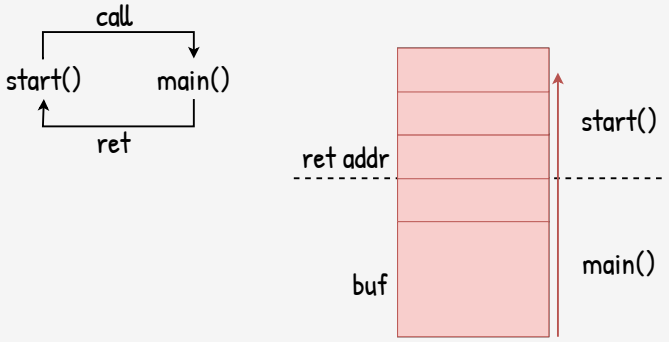
Luego se ejecuta `ret` que toma del stack la `ret` address y salta a dicha dirección. Esto es lo que permite que una función "retorne" a la función llamante (`puts` vuelve a `main`)

El código que restaura los registros y vuelve a la función padre se llama el *epílogo* de `puts`.



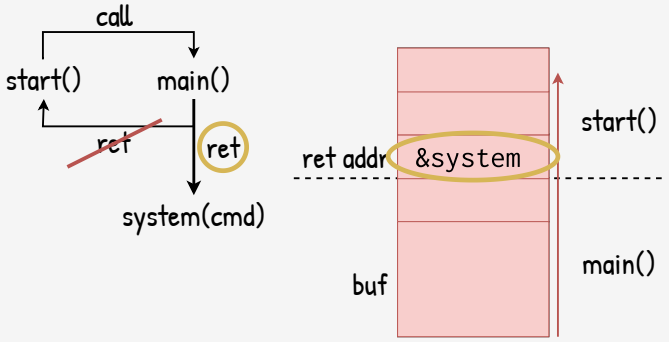
`gets` puede hacer un buffer overflow del buffer `buf` que esta en `main`. Resulta que `main` no es la primer función de un programa, sino `start`.

Es `start` quien llama a `main` y por lo tanto, la `ret` address, el `bp` y otras cosas se encuentran en el stack.

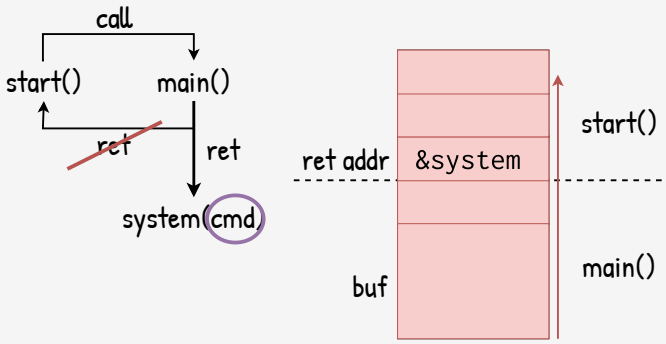


Y al hacer el overflow podemos pisar con el contenido a nuestra elección todo el stack.

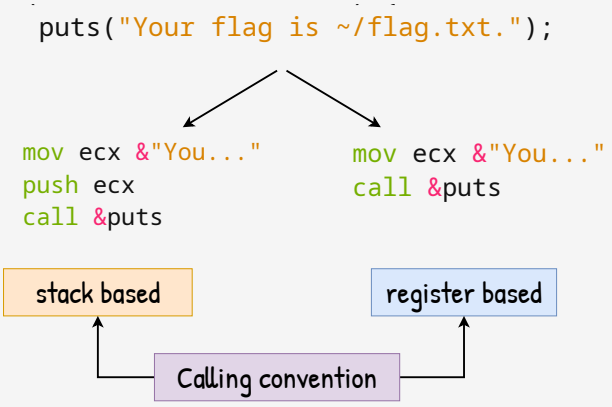
En general esto termina en un crash pero si elegimos correctamente los valores podemos tener un efecto bien definido y mucho más útil q un crash.



Basta con poner *otra* dirección de retorno para que `main`, cuando termine, no salte de vuelta a su padre (`start`) sino a una función q nosotros querramos (por ejemplo `system`)



Y como le pasamos los argumentos a `system`? Nos gustaría que ejecute algo como `system("cat ~/flag.txt")` para q nos revele el flag.

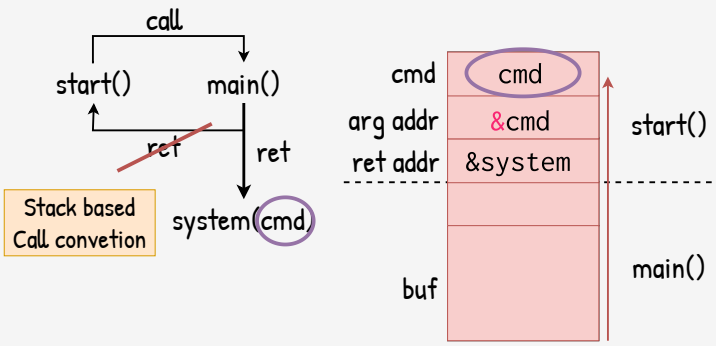


Hay muchas formas de como una función llama a otra. A grandes rasgos puede ser pasando los argumentos por stack o por registros.

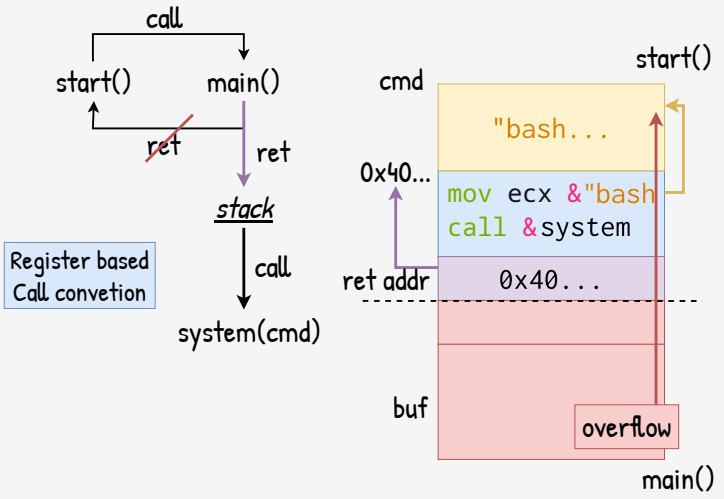
Depende de la arquitectura, del compilador, del OS, si el llamado es hacia una función o no interna.

Para intel/amd de 32 bits en Windows, con el compilador sin activar las optimizaciones, se pasan los argumentos por stack. En 64 bits, por registro. En Linux, 32 o 64 bits, por registro. En. . . .

Esto es la *calling convention* de un programa.



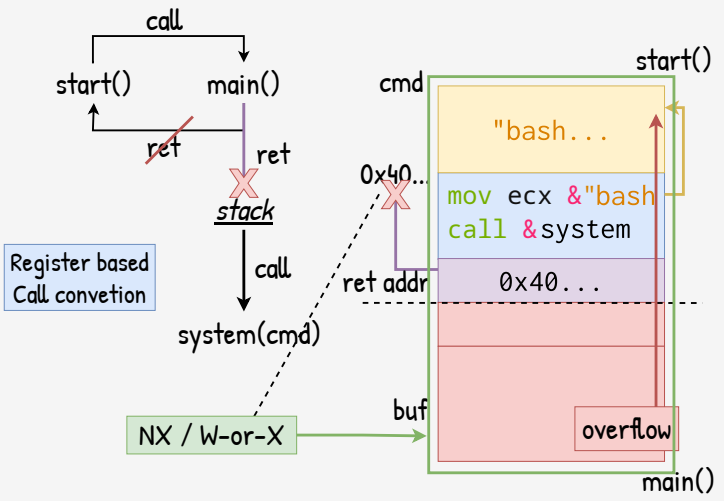
Si la calling convention dice q los argumentos de una función deben ir en el stack, el hack es trivial: con el buffer overflow ya se controla el contenido del stack, solo tenemos q poner la address del argumento (`char*`) y luego el string en si.



Si la calling convention dice q los argumentos de una función deben ir por registro, de alguna manera el programa debe cargarlos por nosotros.

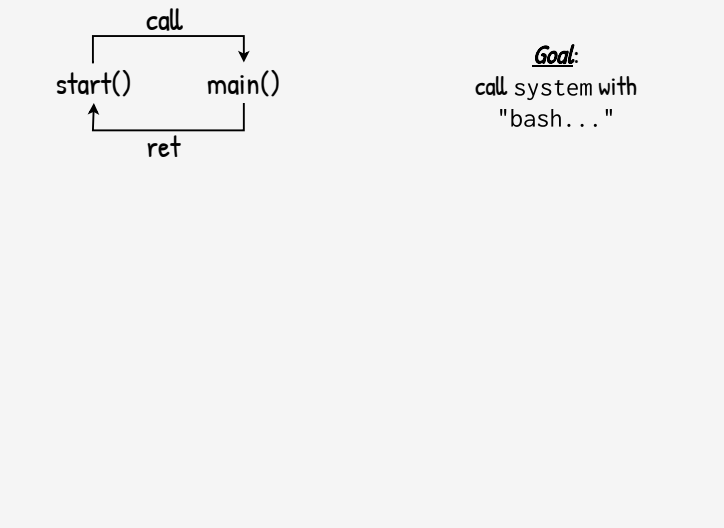
El hack es más sublime: con el overflow se escribe *código ejecutable en el stack* que haga la carga de los registros (`ecx` en este caso) y salte a `system`, tomando como valores los guardados en el mismo stack.

Estamos usando al stack para guardar data y a la vez código ejecutable (en las arquitecturas de Von Neuman, nunca se hizo la separación!)

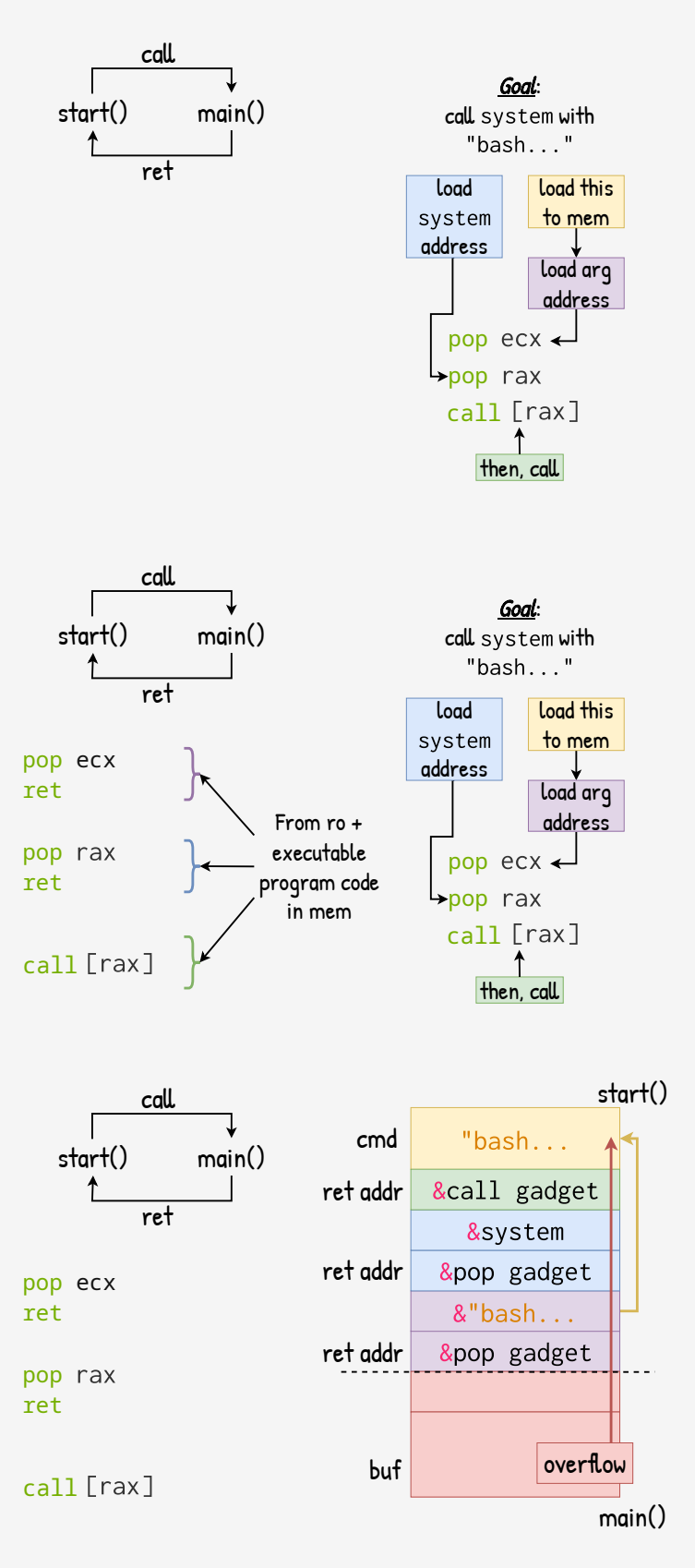


Pero con soporte del hardware y del OS, ejecutar código del stack esta deshabilitado.

La defensa se llama NX (no exec) o W-or-X (write or exec): una página de memoria puede o tener permisos de ejecución (y ser read-only) o permisos de escritura pero no ambos.



Vuelta al principio: queremos llamar a `system` con el string `"bash...."` (o `"cat ..."`)



Hay 4 componentes:

- tener los valores de los argumentos en memoria (fácil, con el overflow podemos escribir en el stack)
- cargar la address del argumento en el registro (`ecx`)
- cargar la address de la función que queremos llamar (`system`) en un registro (`rax`)
- y luego hacer un call indirecto `call [rax]`

Estos 2 últimos componentes parecen innecesarios ya q con el overflow podemos poner la address que querramos en el stack y cuando la func en ejecución termine, ella sola va saltar ahi (asi es como fueron todos los ejemplos anteriores).

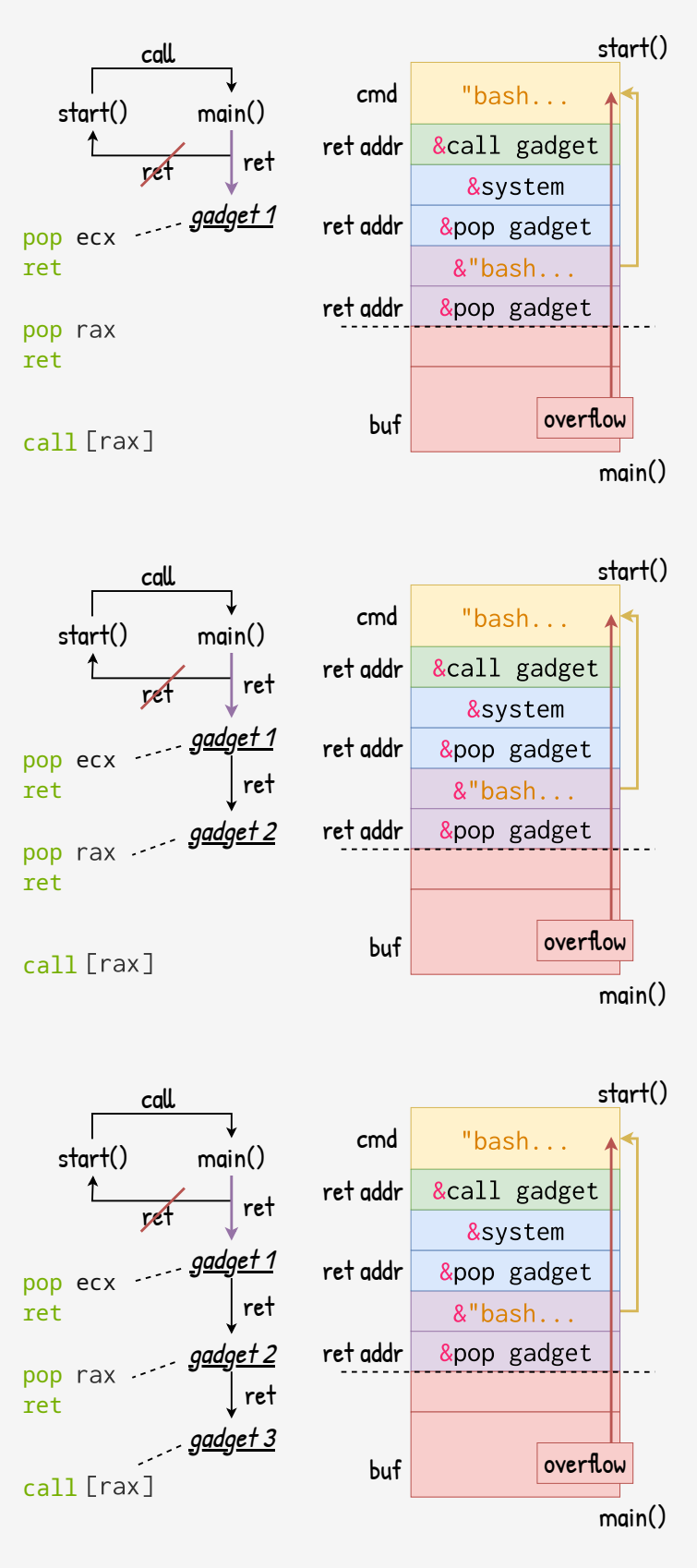
Pero el uso de un call indirecto es más general.

No vamos a encontrar en el codigo del programa algo que haga exactamente lo q necesitamos pero si podemos encontrar fragmentos de codigo que terminen en un `ret` y que tengan side-effects q necesitamos (como sacar algo del stack y ponerlo en `ecx`).

Si llamasemos a cada uno de esto fragmentos tendriamos el efecto final buscado: poner el argumento en `ecx` y llamar a `system`.

Estos fragmentos se llaman *gadgets*.

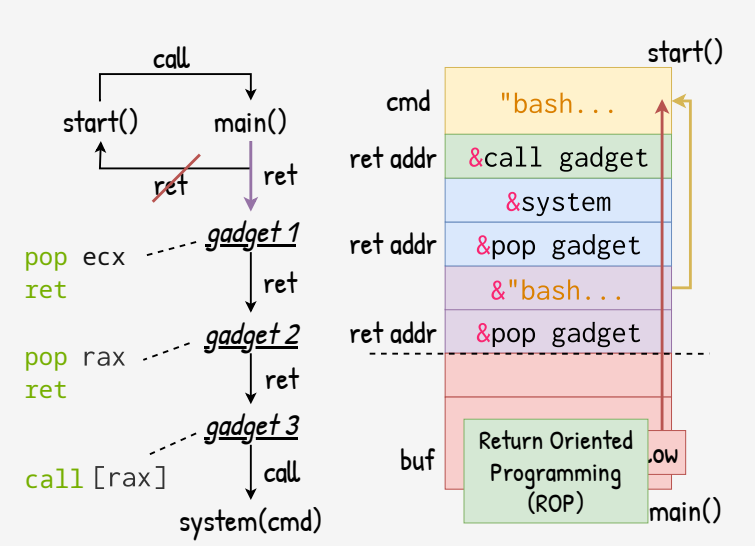
Con el overflow uno debe poner las ret addresses (plural) de cada gadget que se quiere ejecutar junto con sus argumentos.



Cuando `main` termina, ejecuta `ret` y en vez de volver a `start` salta a nuestro primer gadget.

Este toma del stack la address del argumento y lo guarda en `ecx` y ejecuta otro `ret`.

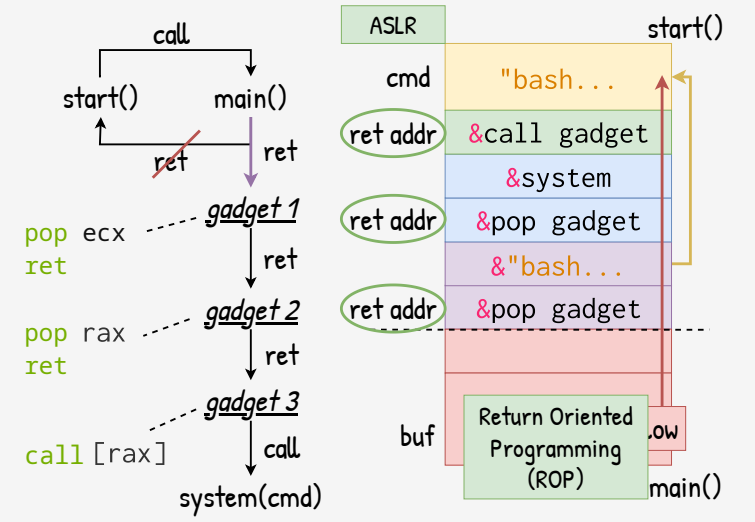
Esto es imporante: todo gadget debe terminar en un `ret` para que pueda leer del stack la address del siguiente gadget y saltar a él.



Esta técnica se llama ROP (*Return Oriented Programming*) y permite ejecutar código, no por que el atacante pueda injectarlo sino reutilizando gadgets existentes.

Una analogía seria: dado un libro, lee una frase específica, frase que no existe en él. Imposible? No!, solo tenes que saltar de página en página leyendo de a una palabra a la vez. Al final "leiste" una frase que no estaba en el libro.

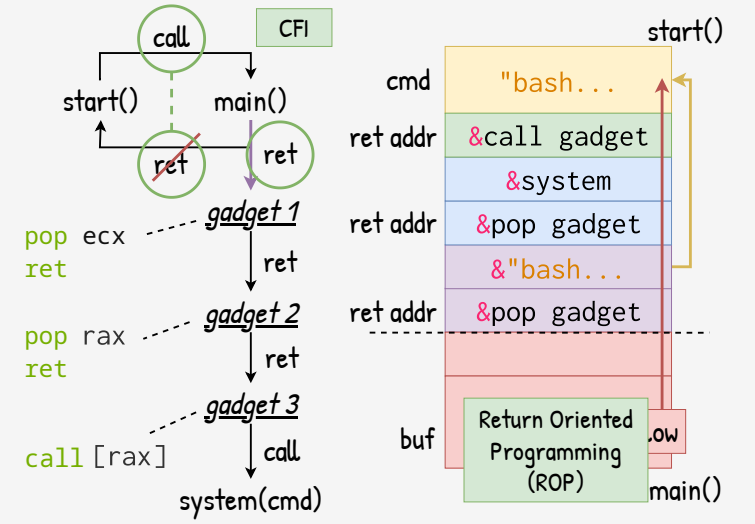
ROP es exactamente lo mismo.



Defensas? ASLR (*Address Space Layout Randomization*): todos los ataques requieren que el adversario sepa las direcciones de los gadgets y las funciones que quiere llamar.

Cuando un programa se carga en memoria, el OS (el loaded) con ASLR activado va a cargarlo en direcciones distintas en cada carga.

Para que el atacante pueda hacer ROP va a tener que combinar el overflow con algun *otro* bug q leakee alguna address del programa para que pueda deducir a partir de el las addresses de los gadgets (y q ese otro bug no te crashee el programa por que si se vuelve a cargar, va a hacerlo en otra dirección aleatoria)

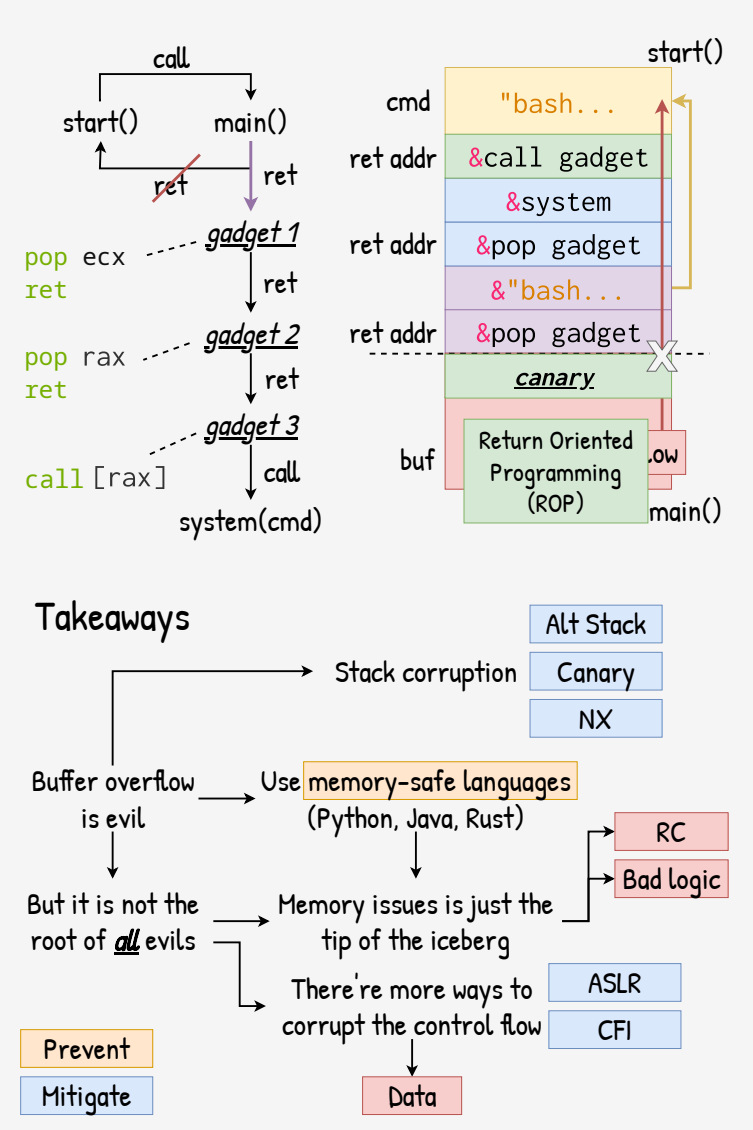


Otra defensa, CFI (*Control Flow Integrity*): El programa registra antes de cada `call` la dirección de retorno que la función debería volver con el `ret` asociado.

Si el `ret` salta a otro lugar, el programa aborta.

Esto mata de raíz ROP pero tiene un costo de performance alto (hay un costo no trivial en cada `call`)

CFI (o defensas similares) estan activadas solo en algunas pocas aplicaciones. ASLR, en cambio, tiene un costo 0 y es el default en la mayoría de los setups.



Una defensa más: canary. La idea guardar en el stack un valor aleatorio secreto al momento de llamar a una función (en el preludio) y se queda una copia en un registro.

Cuando la función está por terminar (en el epílogo), el código compara la cookie (canary) del stack con la del registro y si difieren se asume q hubo un buffer overflow y aborta el programa.